HEART DISEASE PREDICTION

An End-to-End MLOps Implementation

AIMLCZG523 — MLOps Assignment 01

Reproducible ML, MLflow, automated testing, FastAPI, Docker, Kubernetes and monitoring

Project repository

Prepared: 19 October 2026

Abstract

This report documents an educational binary-classification MLOps project using the UCI Cleveland Heart Disease dataset. The workflow includes EDA, reusable preprocessing, model comparison, MLflow tracking, automated tests, FastAPI inference, Docker, Kubernetes manifests and Prometheus monitoring. Logistic Regression was selected by mean five-fold CV ROC-AUC. The model is not clinically validated and must not be used for diagnosis.

1. Objectives and scope

The project demonstrates lifecycle engineering around a classifier: data inspection, training, model persistence, API validation, CI checks, container packaging, deployment configuration and operational metrics. The repository contains source, tests, model artifacts, EDA figures, deployment manifests and evidence screenshots.

2. Dataset and target

The supplied heart.csv contains 303 rows and 14 columns. The 13 predictors are age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca and thal. The binary target is num > 0. Missing values were recorded for ca (4) and thal (2). Numeric features use median imputation and scaling; categorical features use most-frequent imputation and one-hot encoding.

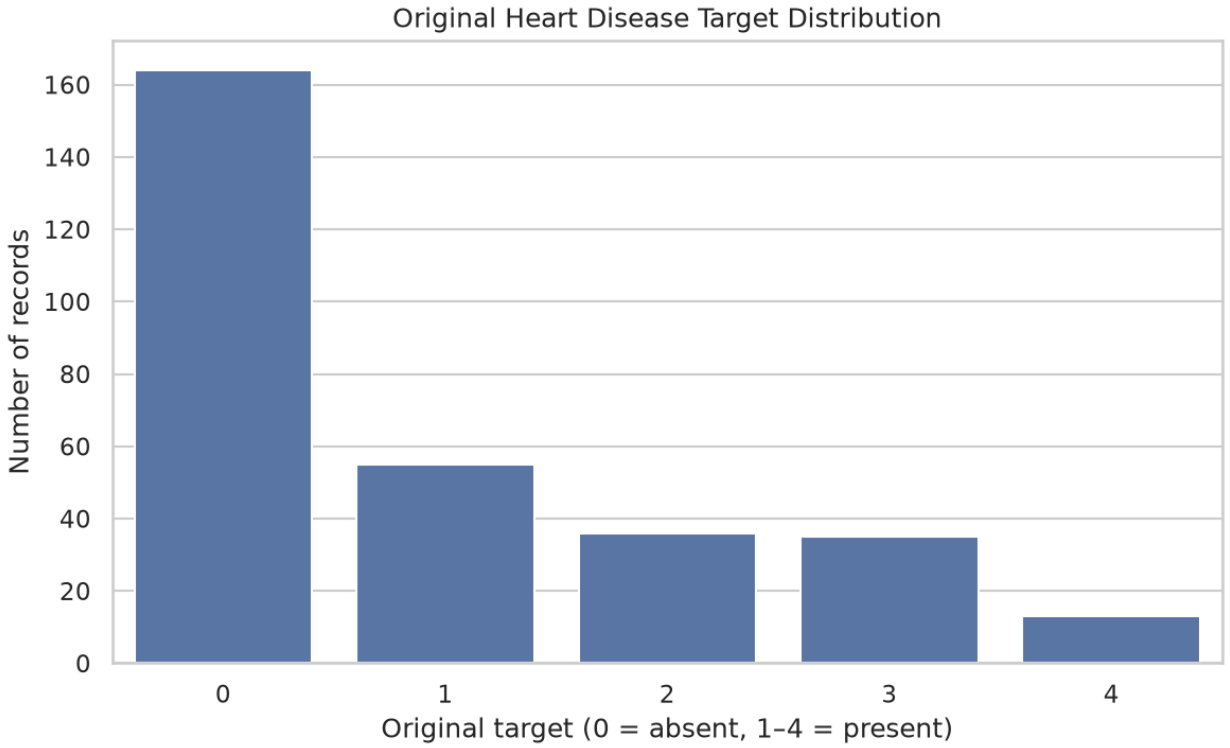


Figure 1. Original target distribution.

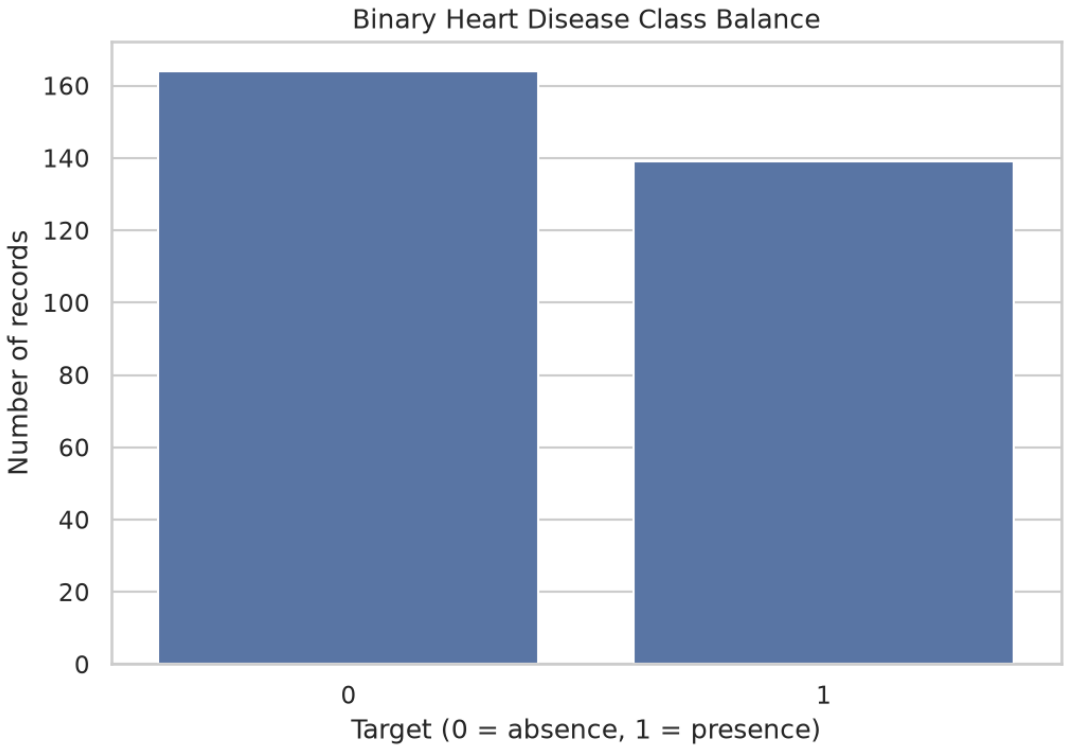


Figure 2. Binary target distribution.

3. Exploratory data analysis

EDA artifacts include target balance, numeric histograms, categorical distributions and a correlation heatmap. They help identify class balance, ranges and associations; correlation does not establish causality.

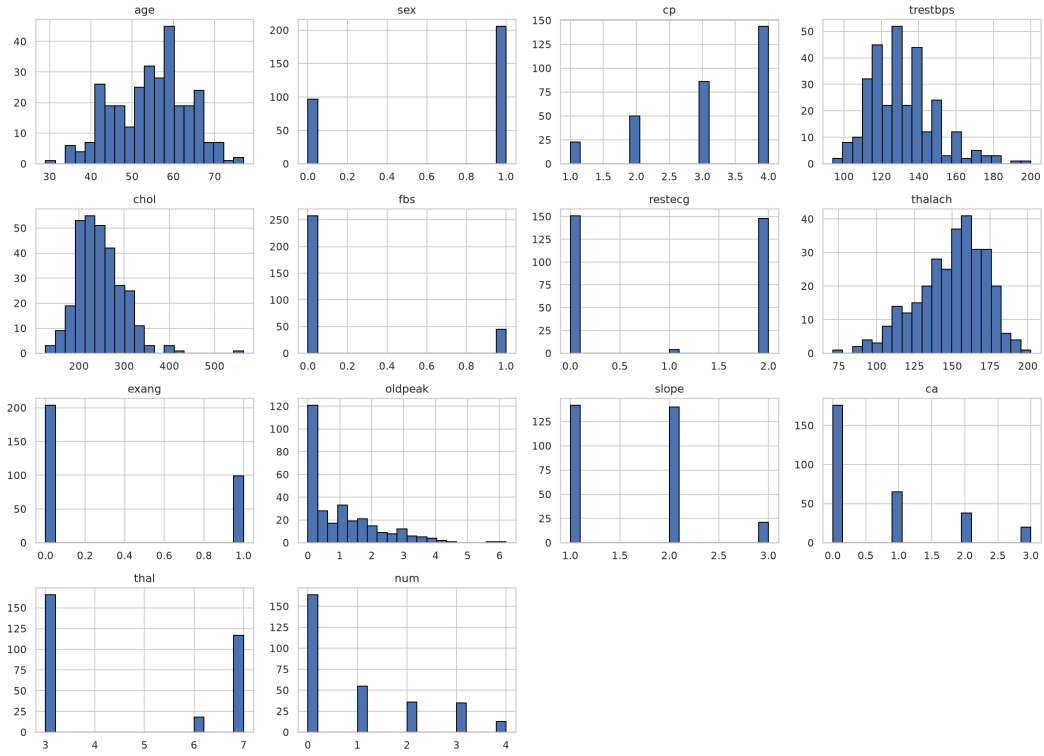


Figure 3. Numeric feature histograms.

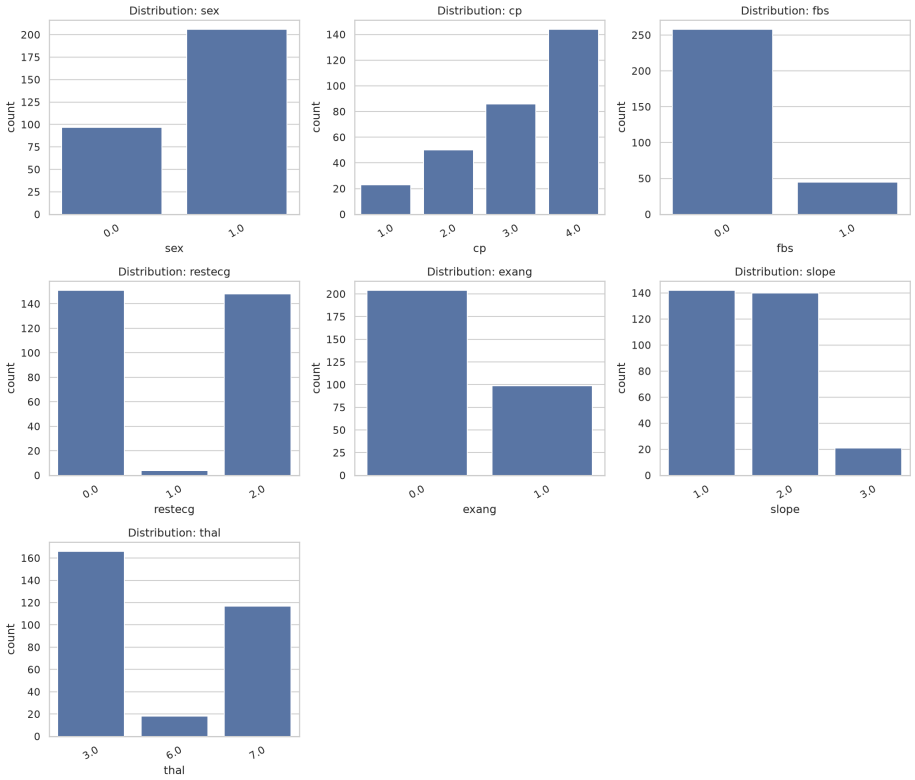


Figure 4. Categorical distributions.

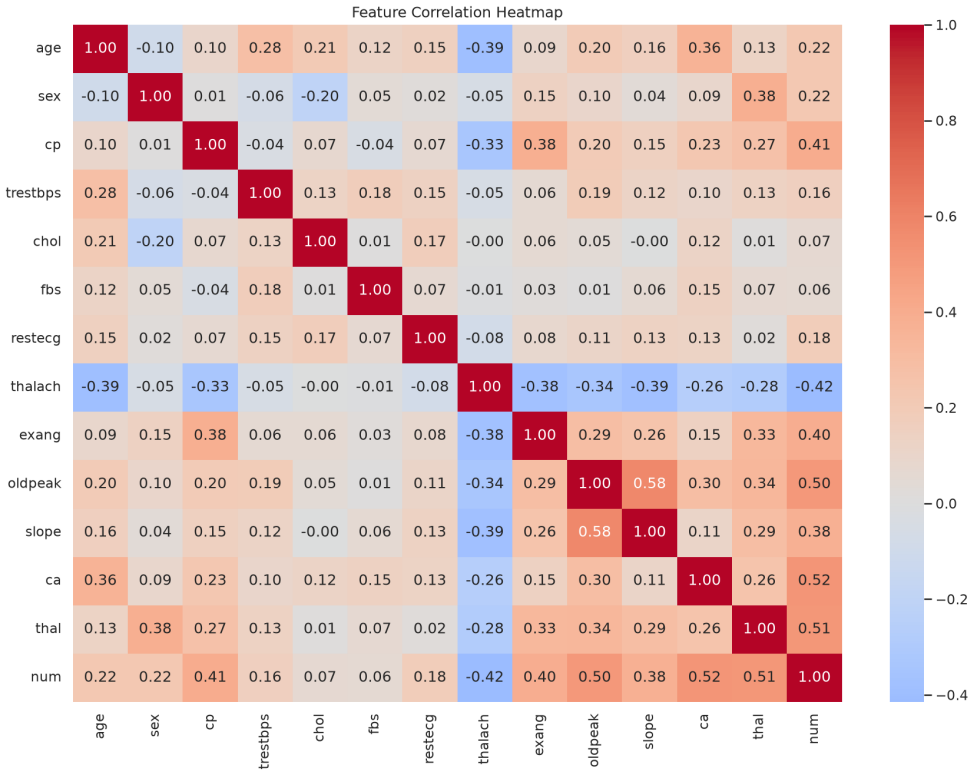


Figure 5. Correlation heatmap.

4. Preprocessing and model development

A ColumnTransformer keeps preprocessing consistent between training and inference. Logistic Regression and Random Forest were evaluated with GridSearchCV and stratified five-fold cross-validation. Metrics include accuracy, precision, recall and ROC-AUC; cross-validation means and standard deviations are saved in model_comparison.csv.

The selected Logistic Regression configuration uses C=1.0. The selected Random Forest configuration uses 200 trees, maximum depth 5 and minimum leaf size 1. Selection was based on mean cross-validation ROC-AUC.

5. Evaluation

Metric	Logistic Regression	Random Forest
Test accuracy	88.52%	88.52%
Test precision	83.87%	83.87%
Test recall	92.86%	92.86%
Test ROC-AUC	0.9665	0.9513
Mean CV ROC-AUC	0.9025	0.9020
CV ROC-AUC std	0.0144	0.0247

Both classifiers achieved 88.52% accuracy, 83.87% precision and 92.86% recall on the held-out test set. Logistic Regression achieved ROC-AUC 0.9665, compared with 0.9513 for Random Forest. Mean CV ROC-AUC was 0.9025 versus 0.9020, a small difference that does not establish general superiority.

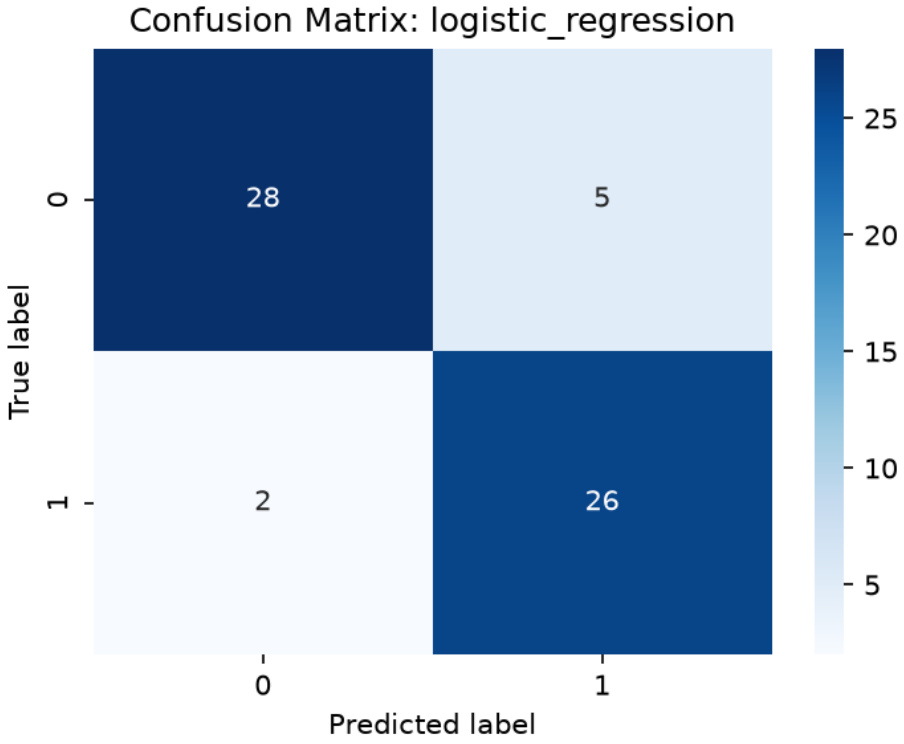


Figure 6. Logistic Regression confusion matrix.

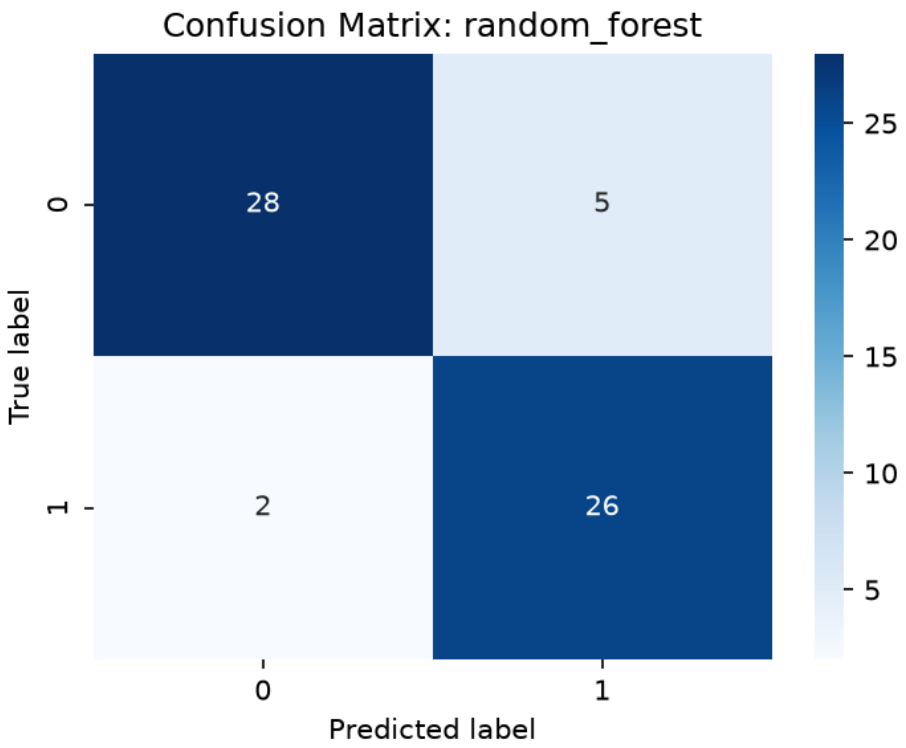


Figure 7. Random Forest confusion matrix.

6. MLflow experiment tracking

MLflow records run parameters, metrics and artifacts, supporting inspection of experiments. Tracking data is stored locally in the project database and artifact directory.

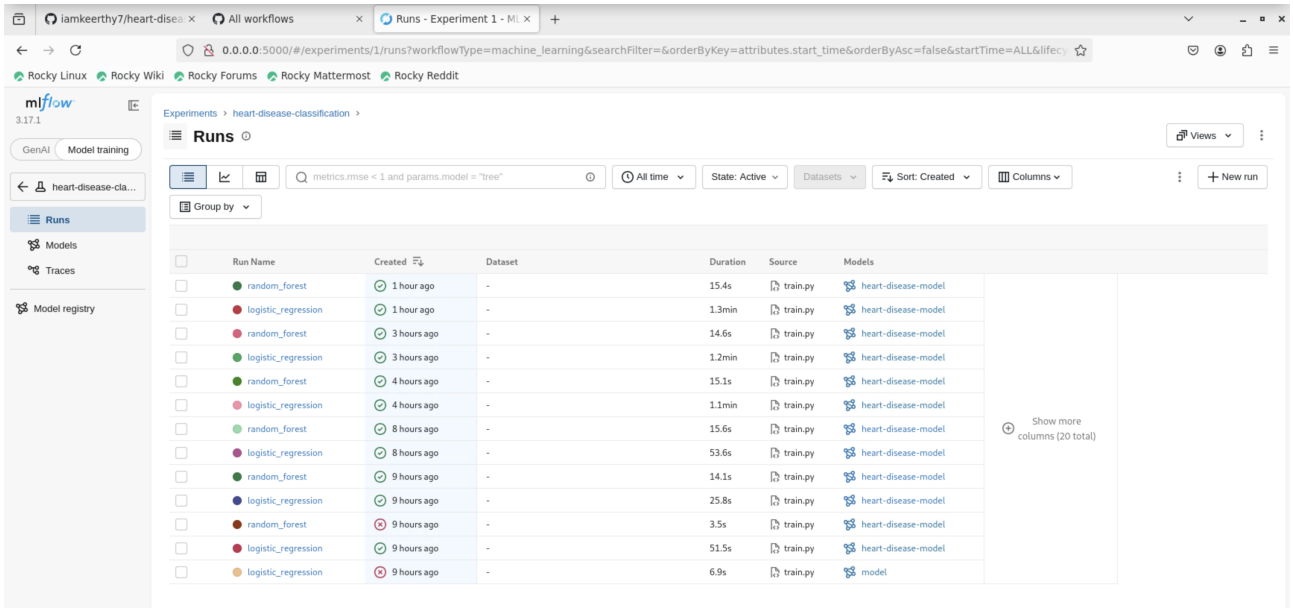


Figure 8. MLflow experiment runs.

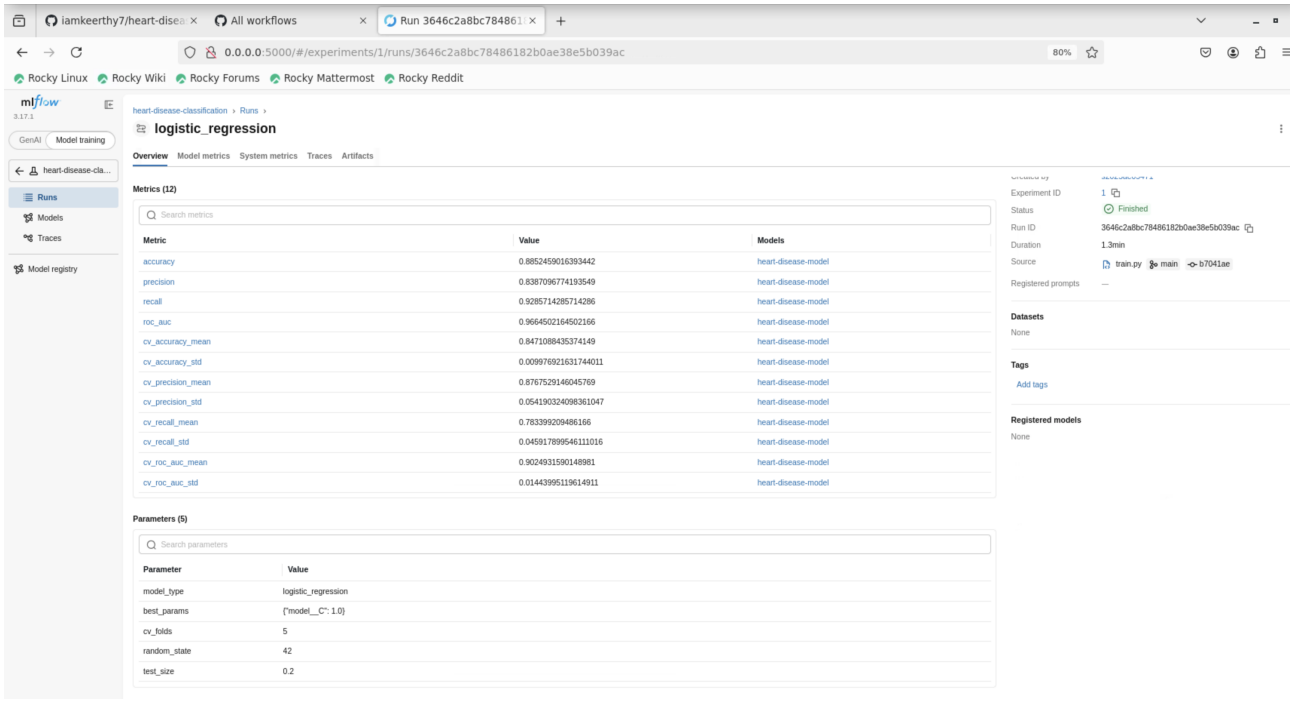


Figure 9. MLflow run details.

7. Model persistence and API

The selected pipeline is stored as models/best_model.joblib; models/model_metadata.json records the selected model, metric, target definition and feature list. FastAPI exposes health, prediction, metrics and dashboard endpoints. Predictions include class, label, confidence, probabilities and an educational-use disclaimer.



Figure 10. Container API prediction.

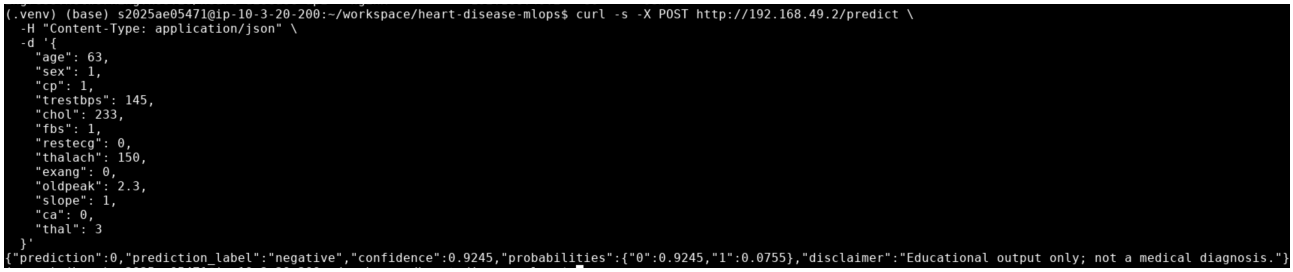


Figure 11. Prediction through Kubernetes ingress.

8. Testing and CI

The project has tests for API health, valid predictions, invalid age, missing features, metrics and dashboard behavior, plus preprocessing checks for finite values, missing data, unseen categories and feature partitioning. Recorded verification was 10 passing tests; Ruff and compile checks also passed. GitHub Actions runs validation, tests and training, then uploads artifacts.

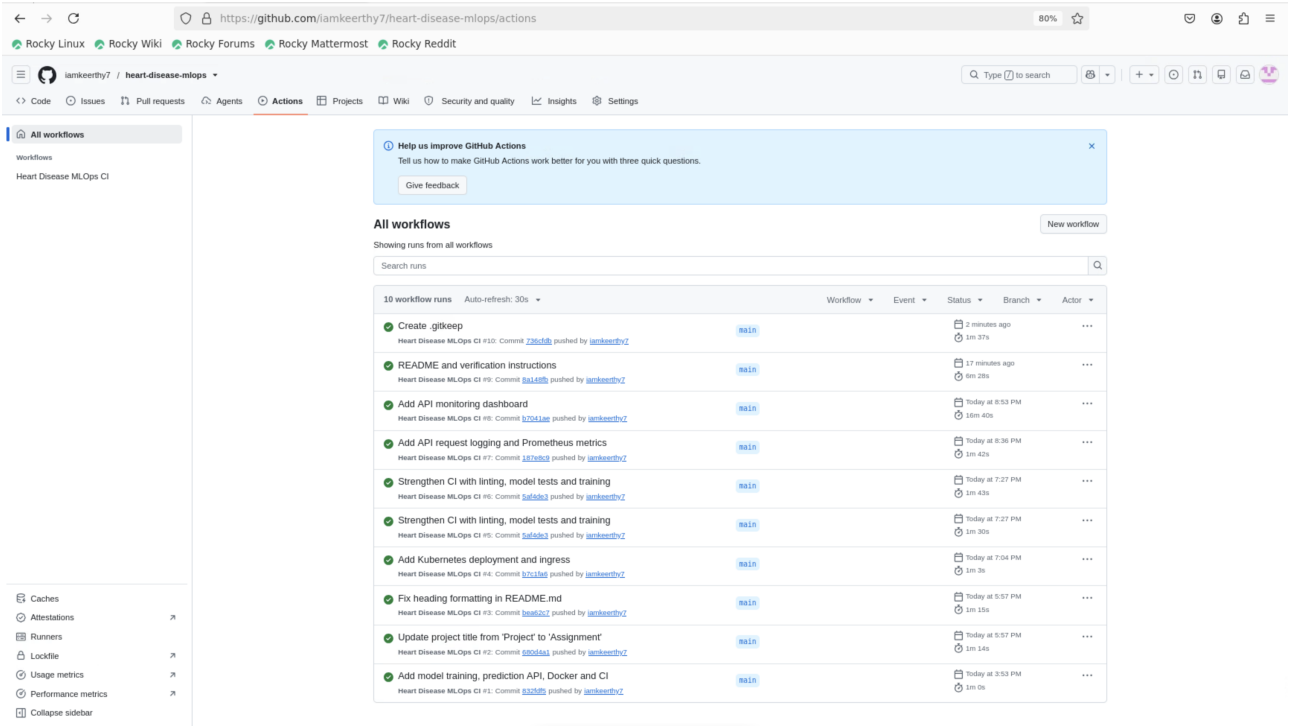


Figure 12. GitHub Actions evidence.

9. Docker and Kubernetes

The Docker image packages the API and model and exposes port 8000. The container was tested on host port 8001 because port 8000 was occupied. Kubernetes manifests configure a Deployment, LoadBalancer Service and NGINX Ingress. Local Minikube showed a ready replica and successful tests of health, prediction, metrics and dashboard routes. The local ingress address is not a public endpoint.

```
(.venv) (base) s2025ae05471@ip-10-3-20-200:~/workspace/heart-disease-mlops$ docker images heart-disease-api
REPOSITORY          TAG       IMAGE ID       CREATED       SIZE
heart-disease-api   1.0       30ec77e7cb08   2 hours ago   1.11GB
(.venv) (base) s2025ae05471@ip-10-3-20-200:~/workspace/heart-disease-mlops$
```

Figure 13. Docker build evidence.

```
(.venv) (base) s2025ae05471@ip-10-3-20-200:~/workspace/heart-disease-mlops$ minikube kubectl -- get pods,services,ingress
NAME                                 READY   STATUS    RESTARTS   AGE
pod/heart-disease-api-56dfb7b7d7-5lxpw   1/1     Running   0          93m

NAME                         TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/heart-disease-api    LoadBalancer   10.109.75.115   <pending>     80:32116/TCP   93m
service/kubernetes           ClusterIP      10.96.0.1       <none>        443/TCP        98m

NAME                                        CLASS   HOSTS   ADDRESS        PORTS   AGE
ingress.networking.k8s.io/heart-disease-api   nginx   *       192.168.49.2   80      93m
```

Figure 14. Kubernetes resources.

10. Monitoring and logging

Request middleware logs method, path, status and duration. Prometheus counters and latency histograms capture HTTP activity and prediction outcomes. The dashboard summarizes request volume, successful predictions, errors, latency and service status.

```
(.venv) (base) s2025ae05471@ip-10-3-20-200:~/workspace/heart-disease-mlops$ curl -s http://192.168.49.2/metrics | grep -E '^# HELP heart_disease_|^heart_disease_http_reque
sts_total|^heart_disease_http_request_duration_seconds_(count|sum)|^heart_disease_predictions_total'
# HELP heart_disease_http_requests_total Total HTTP requests
heart_disease_http_requests_total{endpoint="/health",method="GET",status="200"} 894.0
heart_disease_http_requests_total{endpoint="/dashboard",method="GET",status="200"} 2.0
heart_disease_http_requests_total{endpoint="/metrics",method="GET",status="200"} 11.0
heart_disease_http_requests_total{endpoint="/predict",method="POST",status="200"} 3.0
# HELP heart_disease_http_requests_created Total HTTP requests
# HELP heart_disease_http_request_duration_seconds HTTP request duration in seconds
heart_disease_http_request_duration_seconds_count{endpoint="/health",method="GET"} 894.0
heart_disease_http_request_duration_seconds_sum{endpoint="/health",method="GET"} 0.8866123400052857
heart_disease_http_request_duration_seconds_count{endpoint="/dashboard",method="GET"} 2.0
heart_disease_http_request_duration_seconds_sum{endpoint="/dashboard",method="GET"} 0.0020385620000524796
heart_disease_http_request_duration_seconds_count{endpoint="/metrics",method="GET"} 11.0
heart_disease_http_request_duration_seconds_sum{endpoint="/metrics",method="GET"} 0.04074988400316215
heart_disease_http_request_duration_seconds_count{endpoint="/predict",method="POST"} 3.0
heart_disease_http_request_duration_seconds_sum{endpoint="/predict",method="POST"} 0.1454144749986881
# HELP heart_disease_http_request_duration_seconds_created HTTP request duration in seconds
# HELP heart_disease_predictions_total Prediction requests by outcome
heart_disease_predictions_total{outcome="success"} 3.0
# HELP heart_disease_predictions_created Prediction requests by outcome
```

Figure 15. Prometheus metrics.

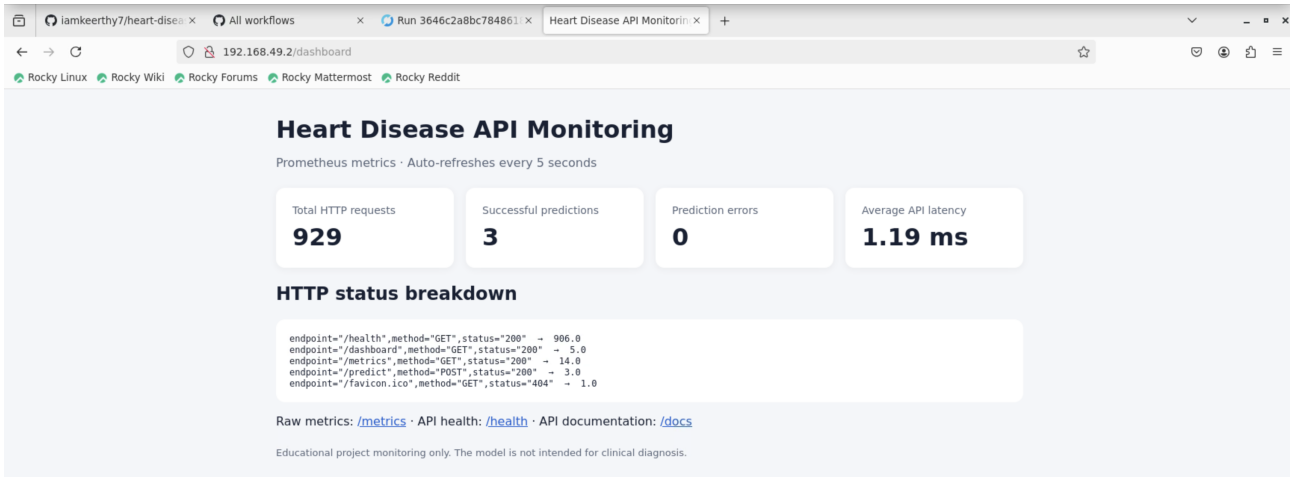


Figure 16. Monitoring dashboard.

11. Reproducibility

Key repository paths are data/heart.csv, src/heart_mlops/, tests/, models/, reports/figures/, deployment/k8s/, .github/workflows/ and screenshots/. Follow README.md for environment setup, training, tests, MLflow, API, Docker and Kubernetes commands. Preserve local MLflow records and model artifacts when reproducing experiment history.

12. Limitations and future work

The dataset is small (303 observations) and historical. A held-out split and cross-validation do not establish clinical validity, calibration, fairness or generalization to other populations. Future work includes external validation, calibration, threshold analysis, drift monitoring, authentication, rate limiting, image/dependency scanning, secrets management and secure cloud deployment.

Architecture

The diagram summarizes the data, training, experiment tracking, API serving, CI/CD, container deployment and monitoring workflow.

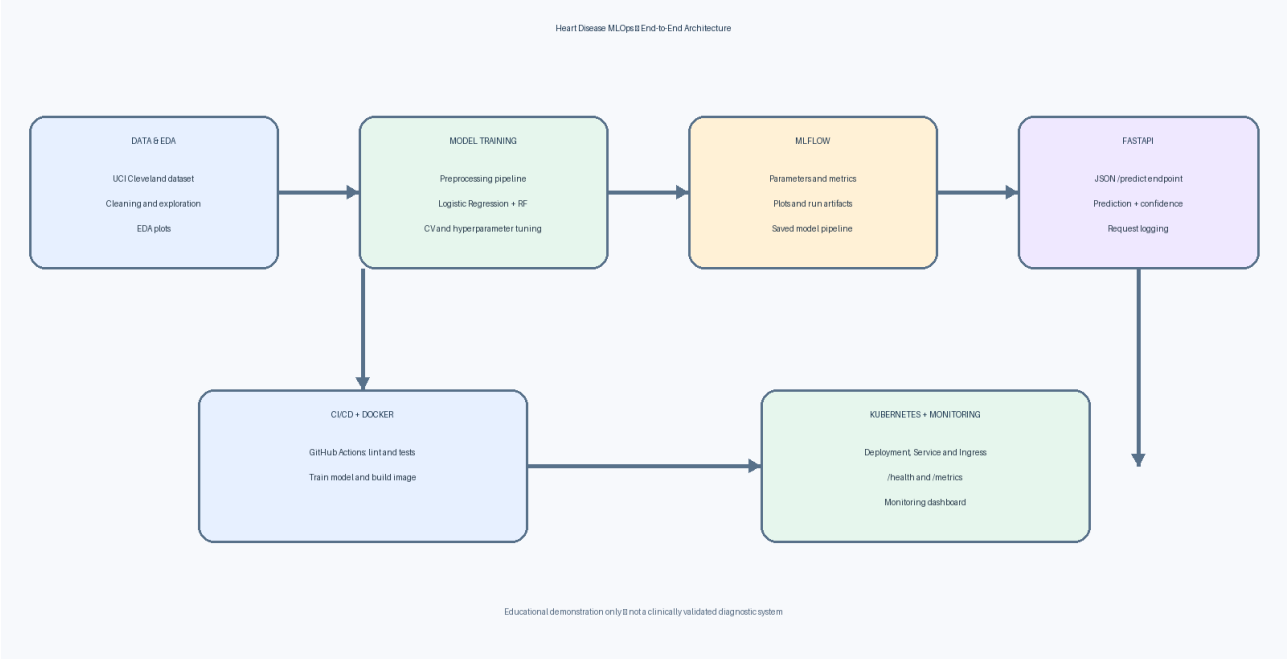


Figure 17. End-to-end architecture of the implemented MLOps workflow.

Conclusion

The implementation demonstrates the main MLOps stages from data analysis to monitored API serving. Logistic Regression was selected based on mean cross-validation ROC-AUC. Results apply to this dataset and split only; this system is an educational demonstration, not medical advice.

Repository and evidence

https://github.com/iamkeerthy7/heart-disease-mlops

Evidence screenshots are under screenshots/; EDA and confusion matrix figures are under reports/figures/. Consult README.md for commands and setup.